

DESAFÍO TÉCNICO DE PRÁCTICA

Construcción de soluciones sobre Sparx Enterprise Architect y Prolaborate

Pasantías Proagile 2026



Informe Ejercicio 2 - Queries



Ezequiel Pino

Enfoque de solución	3
Identificación de aplicaciones	4
Consulta Q1-C—Cantidad de aplicaciones con categoría ORO	4
Consulta Q1-L—Listado de aplicaciones con categoría ORO	6
Consulta Q2-G—Aplicaciones por estado de Vigencia	7
Consulta Q3-C—Cantidad de aplicaciones afectadas por Base de Datos 28	9
Consulta Q3-L—Aplicaciones afectadas por Base de Datos 28	11
Exclusión de elementos no aplicación	12
Uso de Direction	13
Estrategia de unicidad	13
Validación de las consultas	14
Visualización opcional en Prolaborate	15
Objetivo de la visualización	15
Consulta utilizada como referencia funcional	16
Configuración inicial mediante consulta SQL	17
Dificultad encontrada	17
Configuración del gráfico de dona	19
Configuración del gráfico de barras	20
Resultado del dashboard	21
Evidencia funcional	22
Documentación técnica complementaria	22

Informe Principal – Query Prolaborate y Estadísticas sobre EA

El presente documento describe la solución desarrollada para el segundo ejercicio del desafío técnico de Proagile, orientado a la explotación de información almacenada en un repositorio de Sparx Enterprise Architect mediante consultas SQL.

El objetivo del ejercicio consiste en responder preguntas de gobierno de aplicaciones relacionadas con categoría, vigencia y análisis de impacto, utilizando la estructura interna del repositorio de Enterprise Architect y validando los resultados obtenidos contra los elementos modelados.

La solución fue diseñada para ser completamente de solo lectura. Las consultas no modifican el repositorio y se limitan a operaciones SELECT.

La validación se realizó en dos niveles. En primer lugar, ejecuté diagnósticos sobre el archivo .qea mediante SQLite en modo read-only, utilizados únicamente como mecanismo auxiliar de comprobación. Posteriormente, ejecuté cada consulta final de forma manual en Enterprise Architect mediante SQL Search, siendo esta última la superficie considerada válida para la aceptación funcional.

Las evidencias detalladas de ejecución se encuentran en el documento complementario:

[Pino_Evidencias_Funcionales_Queries.pdf](#)

El registro de uso de herramientas de Inteligencia Artificial se encuentra en:

[Pino_Registro_Uso_IA_Ejercicio2.pdf](#)

Enfoque de solución

El ejercicio plantea tres interrogantes de negocio:

- identificar las aplicaciones cuya categoría es **ORO**;
- conocer la cantidad de aplicaciones en estado **Vigente** y **Deprecado**;
- determinar qué aplicaciones se verían afectadas ante la baja de **Base de Datos 28**.

A partir de estas preguntas, luego de analizar, se definieron cinco consultas SQL finales:

Q1-C — Categoría ORO — Count

Q1-L — Categoría ORO — List

Q2-G — Vigencia — Grouped

Q3-C — DB28 Impact — Count

Q3-L — DB28 Impact — List

Decidí utilizar cinco consultas en lugar de tres con el objetivo de separar resultados agregados de resultados de detalle. De esta forma, busqué que las consultas de conteo producen una única fila agregada, mientras que las consultas de listas permiten navegar desde el resultado hacia los elementos reales del modelo en Enterprise Architect.

El diseño aprobado utiliza exactamente esta distribución de cinco **SELECT**: dos para la primera pregunta, uno para la segunda y dos para la tercera.

Todas las consultas se almacenan en:

[Pino_Exercise_2_EA_Governance_Queries.sql](#)

El archivo funciona únicamente como contenedor de entrega. Las consultas fueron creadas y ejecutadas individualmente en Enterprise Architect y no como un único batch.

Identificación de aplicaciones

Las tres preguntas requieren identificar correctamente qué elementos del repositorio representan aplicaciones. Para ello se utilizó el siguiente criterio:

```
Object_Type = 'Class'
AND Stereotype = 'ArchiMate_ApplicationComponent'
```

El filtro sobre Object_Type por sí solo no resulta suficiente, debido a que el repositorio también contiene otros elementos representados físicamente como Class, por ejemplo Data Objects y Business Actors.

El estereotipo permite distinguir específicamente los elementos correspondientes a aplicaciones.

Este criterio se aplica tanto en las consultas de categoría y vigencia como en el origen de las dependencias utilizadas para el análisis de impacto.

Consulta Q1-C—Cantidad de aplicaciones con categoría ORO

La primera parte de la consulta responde:

¿Cuántos elementos de tipo aplicación tienen el tagged value Categoría con valor ORO?

La información principal del elemento se obtiene desde:

t_object

mientras que los tagged values se encuentran almacenados en:

t_objectproperties

Ambas tablas se relacionan mediante:

tag.Object_ID = app.Object_ID

Los filtros utilizados son:

```
app.Object_Type = 'Class'
app.Stereotype = 'ArchiMate_ApplicationComponent'
tag.Property = 'Categoria'
tag.Value = 'ORO'
```

La consulta utiliza:

COUNT(DISTINCT app.Object_ID)

en lugar de contar directamente las filas generadas por el JOIN.

Esto evita que una eventual duplicación de tagged values haga que la misma aplicación sea contabilizada más de una vez.

La consulta final es:

```
SELECT
    COUNT(DISTINCT app.Object_ID) AS ORO_Application_Count
FROM t_object app
INNER JOIN t_objectproperties tag
    ON tag.Object_ID = app.Object_ID
WHERE app.Object_Type = 'Class'
    AND app.Stereotype = 'ArchiMate_ApplicationComponent'
    AND tag.Property = 'Categoria'
    AND tag.Value = 'ORO';
```

El resultado obtenido en Enterprise Architect fue: 8



Por lo tanto, existen ocho aplicaciones cuya categoría es ORO.

Consulta Q1-L—Listado de aplicaciones con categoría ORO

La segunda consulta responde a la parte:

¿Cuáles son las aplicaciones con categoría ORO?

Se utilizan las mismas tablas, relación y filtros que en Q1-C, pero en lugar de generar un agregado se obtiene una fila por aplicación.

La consulta utiliza **SELECT DISTINCT** para preservar la unicidad lógica de los elementos.

Además se incorporan:

```
app.ea_guid AS CLASSGUID
app.Object_Type AS CLASSTYPE
```

Estas columnas poseen un significado especial para Enterprise Architect SQL Search y permiten que las filas retornadas puedan asociarse nuevamente con los elementos del modelo. Esto nos permite, por ejemplo, realizar doble clic sobre una aplicación obtenida por la consulta y abrir directamente sus propiedades en Enterprise Architect.

La consulta es:

```
SELECT DISTINCT
  app.ea_guid AS CLASSGUID,
  app.Object_Type AS CLASSTYPE,
  app.Name AS Application_Name,
  tag.Value AS Categoria
FROM t_object app
INNER JOIN t_objectproperties tag
  ON tag.Object_ID = app.Object_ID
WHERE app.Object_Type = 'Class'
  AND app.Stereotype = 'ArchiMate_ApplicationComponent'
  AND tag.Property = 'Categoria'
  AND tag.Value = 'ORO'
ORDER BY app.Name;
```

Se obtuvieron las siguientes aplicaciones:

- Aplicación 1
- Aplicación 2
- Aplicación 3
- Aplicación 4
- Aplicación 6
- Aplicación 8
- Aplicación 20
- Aplicación 29

Todas ellas presentaron **Categoria = ORO**.

<Search Term> Mis búsquedas Q1-L – Categoría ORO – List

Constructor de consulta Bloc de notas SQL

Buscar en: Acción de sub-filtro

```

SELECT DISTINCT
  app.ea_guid AS CLASSGUID,
  app.Object_Type AS CLASSTYPE,
  app.Name AS Application_Name,
  tag.Value AS Categoria
FROM t_object app
INNER JOIN t_objectproperties tag
  ON tag.Object_ID = app.Object_ID
WHERE app.Object_Type = 'Class'
  AND app.Stereotype = 'ArchiMate_ApplicationComponent'
  AND tag.Property = 'Categoria'
  AND tag.Value = 'ORO'
ORDER BY app.Name;

```

Arrastrar un encabezado de la columna aquí para agrupar por esa columna.

	Application_Name	Categoria
	Aplicación 1	ORO
	Aplicación 2	ORO
	Aplicación 20	ORO
	Aplicación 29	ORO
	Aplicación 3	ORO
	Aplicación 4	ORO
	Aplicación 6	ORO

La navegación desde el resultado de SQL Search hacia los elementos del modelo fue validada correctamente.

La separación entre Q1-C y Q1-L evita mezclar en una misma estructura una fila agregada con filas representativas de elementos navegables. Esta decisión mejora tanto la claridad como la trazabilidad de la solución.

Consulta Q2-G—Aplicaciones por estado de Vigencia

La segunda pregunta del desafío es:

¿Cuántas aplicaciones tenemos en cada estado de vigencia: Vigente y Deprecado?

Al igual que Categoría, el valor de Vigencia se encuentra almacenado como tagged value en t_objectproperties.

Por este motivo se utilizan nuevamente:

t_object

t_objectproperties

relacionadas mediante:

tag.Object_ID = app.Object_ID

Se filtra:

`tag.Property = 'Vigencia'`

y únicamente se consideran los estados requeridos:

`tag.Value IN ('Vigente', 'Deprecado')`

Es importante destacar que no se utiliza:

`t_object.Status`

como sustituto del concepto de vigencia.

Status corresponde a otra propiedad del elemento y no representa el tagged value solicitado por el ejercicio.

La consulta utiliza:

`COUNT(DISTINCT app.Object_ID)`

para garantizar que cada aplicación contribuya una única vez al grupo correspondiente.

La consulta final es:

```
SELECT
    tag.Value AS Vigencia,
    COUNT(DISTINCT app.Object_ID) AS Application_Count
FROM t_object app
INNER JOIN t_objectproperties tag
    ON tag.Object_ID = app.Object_ID
WHERE app.Object_Type = 'Class'
    AND app.Stereotype = 'ArchiMate_ApplicationComponent'
    AND tag.Property = 'Vigencia'
    AND tag.Value IN ('Vigente', 'Deprecado')
GROUP BY tag.Value
ORDER BY tag.Value;
```

Los resultados obtenidos en Enterprise Architect fueron:

Deprecado = 14

Vigente = 16

Durante el diagnóstico previo también se verificó que las aplicaciones del repositorio utilizado no presentan valores NULL, vacíos o N/A para Vigencia.

Por este motivo la consulta no genera artificialmente categorías adicionales con valor cero.

También se verificó que ninguna aplicación presenta simultáneamente valores de Vigencia conflictivos. Q2 se resolvió mediante una única consulta agrupada porque la pregunta de

negocio solicita directamente cantidades por estado y no requiere un listado adicional de elementos.

Constructor de consulta Bloc de notas SQL

Buscar en: Acción de sub-filtro

```

SELECT
    tag.Value AS Vigencia,
    COUNT(DISTINCT app.Object_ID) AS Application_Count
FROM t_object app
INNER JOIN t_objectproperties tag
    ON tag.Object_ID = app.Object_ID
WHERE app.Object_Type = 'Class'
    AND app.Stereotype = 'Archimate_ApplicationComponent'
    AND tag.Property = 'Vigencia'
    AND tag.Value IN ('Vigente', 'Deprecado')
GROUP BY tag.Value
ORDER BY tag.Value;

```

Arrastrar un encabezado de la columna aquí para agrupar por esa columna.

Vigencia	Application_Count
Deprecado	14
Vigente	16

Consulta Q3-C—Cantidad de aplicaciones afectadas por Base de Datos 28

La tercera pregunta del desafío es:

¿Qué aplicaciones se verían afectadas si se da de baja la Base de Datos 28?

En este caso se debe analizar la relación entre elementos del repositorio.

Se utilizan tres referencias principales:

t_object src
t_connector rel
t_object tgt

src representa el elemento origen.

rel representa la relación.

tgt representa el elemento destino.

Las uniones utilizadas son:

rel.Start_Object_ID = src.Object_ID
tgt.Object_ID = rel.End_Object_ID

Por lo tanto, la interpretación aplicada es:

Start_Object_ID = origen
End_Object_ID = destino

La relación debe ser:

rel.Connector_Type = 'Dependency'

El origen se limita a aplicaciones mediante:

src.Object_Type = 'Class'
AND src.Stereotype = 'ArchiMate_ApplicationComponent'

El destino se identifica semánticamente mediante:

tgt.Name = 'Base de Datos 28'
AND tgt.Object_Type = 'Class'
AND tgt.Stereotype = 'ArchiMate_DataObject'

Durante el diagnóstico del repositorio se observó que este elemento posee actualmente **Object_ID = 102**.

Sin embargo, ese identificador no se utiliza como filtro en las consultas finales. Se decidió identificar el destino por nombre, tipo y estereotipo para evitar depender de una clave interna que podría cambiar en otro repositorio o después de una importación.

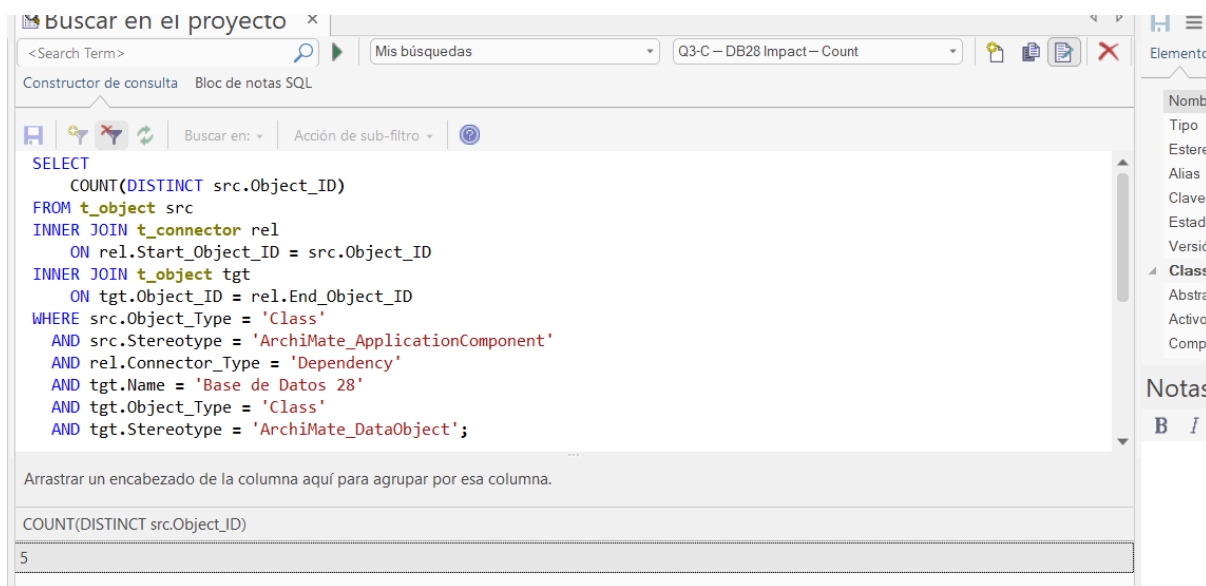
Antes de aceptar la consulta se verificó además que este criterio semántico identifica exactamente un único elemento.

La consulta de conteo es:

```
SELECT
  COUNT(DISTINCT src.Object_ID)
FROM t_object src
INNER JOIN t_connector rel
  ON rel.Start_Object_ID = src.Object_ID
INNER JOIN t_object tgt
  ON tgt.Object_ID = rel.End_Object_ID
WHERE src.Object_Type = 'Class'
  AND src.Stereotype = 'ArchiMate_ApplicationComponent'
  AND rel.Connector_Type = 'Dependency'
  AND tgt.Name = 'Base de Datos 28'
  AND tgt.Object_Type = 'Class'
  AND tgt.Stereotype = 'ArchiMate_DataObject';
```

El resultado obtenido fue: 5

Por lo tanto, cinco aplicaciones se encuentran afectadas por dependencias hacia Base de Datos 28.



Consulta Q3-L—Aplicaciones afectadas por Base de Datos 28

Para conocer cuáles son esas aplicaciones se implementó una segunda consulta. Esta consulta fue la que mayor dificultad me llevó. La lógica de joins y filtros es idéntica a Q3-C, pero el resultado contiene una fila por aplicación origen.

También incorpora:

```
src.ea_guid AS CLASSGUID
src.Object_Type AS CLASSTYPE
```

para habilitar la navegación desde SQL Search hacia la aplicación correspondiente.

La consulta es:

```
SELECT DISTINCT
    src.ea_guid AS CLASSGUID,
    src.Object_Type AS CLASSTYPE,
    src.Name AS Source_Application,
    tgt.Name AS Target_Database
FROM t_object src
INNER JOIN t_connector rel
    ON rel.Start_Object_ID = src.Object_ID
INNER JOIN t_object tgt
    ON tgt.Object_ID = rel.End_Object_ID
WHERE src.Object_Type = 'Class'
    AND src.Stereotype = 'ArchiMate_ApplicationComponent'
    AND rel.Connector_Type = 'Dependency'
    AND tgt.Name = 'Base de Datos 28'
```

```
AND tgt.Object_Type = 'Class'
AND tgt.Stereotype = 'ArchiMate_DataObject'
ORDER BY src.Name;
```

Las aplicaciones obtenidas fueron:

Aplicación 5
Aplicación 12
Aplicación 22
Aplicación 25
Aplicación 27

Todas ellas presentan como destino:

Base de Datos 28

La navegación desde el resultado hacia los elementos de Enterprise Architect fue verificada correctamente.

Además, desde las propiedades del elemento se comprobó visualmente la existencia de una relación de tipo Dependencia con Base de Datos 28.

The screenshot shows the SQL query editor with the following query:

```
SELECT DISTINCT
  src.ea_guid AS CLASSGUID,
  src.Object_Type AS CLASSTYPE,
  src.Name AS Source_Application,
  tgt.Name AS Target_Database
FROM t_object src
INNER JOIN t_connector rel
  ON rel.Start_Object_ID = src.Object_ID
INNER JOIN t_object tgt
  ON tgt.Object_ID = rel.End_Object_ID
WHERE src.Object_Type = 'Class'
AND src.Stereotype = 'ArchiMate_ApplicationComponent'
AND rel.Connector_Type = 'Dependency'
AND tgt.Name = 'Base de Datos 28'
AND tgt.Object_Type = 'Class'
```

Below the query, the results table is displayed with the following data:

	Source_Application	Target_Database
	Aplicación 12	Base de Datos 28
	Aplicación 22	Base de Datos 28
	Aplicación 25	Base de Datos 28
	Aplicación 27	Base de Datos 28
	Aplicación 5	Base de Datos 28

Exclusión de elementos no aplicación

Durante el análisis del repositorio se encontraron también dependencias hacia Base de Datos 28 provenientes de:

Servidor 2
Servidor 5
Servidor 19

Estos elementos corresponden a nodos de infraestructura y no a aplicaciones.

Por este motivo no forman parte del resultado obligatorio.

El filtro:

```
src.Object_Type = 'Class'
AND src.Stereotype = 'ArchiMate_ApplicationComponent'
```

permite excluirlas.

Esta verificación se utilizó como control negativo para comprobar que el análisis de impacto responde específicamente a la pregunta sobre **aplicaciones afectadas**.

Uso de Direction

El repositorio contiene información adicional en el campo:

t_connector.Direction

Sin embargo, esta propiedad no fue utilizada como filtro obligatorio.

La orientación de la dependencia se determina mediante los campos estructurales:

Start_Object_ID
End_Object_ID

junto con:

Connector_Type = Dependency

De esta manera la consulta evita introducir una condición redundante adicional.

Estrategia de unicidad

Las consultas fueron diseñadas para representar aplicaciones únicas y no simplemente filas físicas producidas por los **JOIN**. Para los resultados agregados se utilizó:

COUNT(DISTINCT app.Object_ID)

o:

COUNT(DISTINCT src.Object_ID)

según corresponda.

Para los listados se utilizó:

```
SELECT DISTINCT
```

seleccionando únicamente atributos estables por aplicación.

No se incluyeron identificadores de filas de tagged values ni identificadores internos de conectores que pudieran provocar múltiples filas para una misma aplicación. Esta decisión busca mantener el significado de negocio del resultado incluso si existieran múltiples filas físicas asociadas al mismo elemento.

Validación de las consultas

Las cinco consultas finales fueron validadas mediante dos mecanismos complementarios. La primera validación se realizó directamente sobre el archivo .gea utilizando SQLite abierto en modo read-only. Esta ejecución permitió comprobar los resultados esperados sin modificar el repositorio.

Posteriormente, las cinco consultas fueron ejecutadas manualmente en Enterprise Architect SQL Search.

Los resultados obtenidos fueron:

Consulta	Resultado
Q1-C	8 aplicaciones ORO
Q1-L	8 aplicaciones identificadas
Q2-G	Vigente 16 / Deprecado 14
Q3-C	5 aplicaciones afectadas
Q3-L	Aplicaciones 5, 12, 22, 25 y 27

En las consultas de listado también se verificó la navegación hacia elementos reales utilizando **CLASSGUID** y **CLASSTYPE**.

Para Q1 se comprobaron además los tagged values de categoría.

Para Q2 se verificaron tagged values de vigencia visibles en el modelo.

Para Q3 se comprobó tanto la navegación hacia las aplicaciones como la presencia de relaciones Dependency hacia Base de Datos 28.

Las capturas y detalles de estas pruebas se encuentran documentados en:

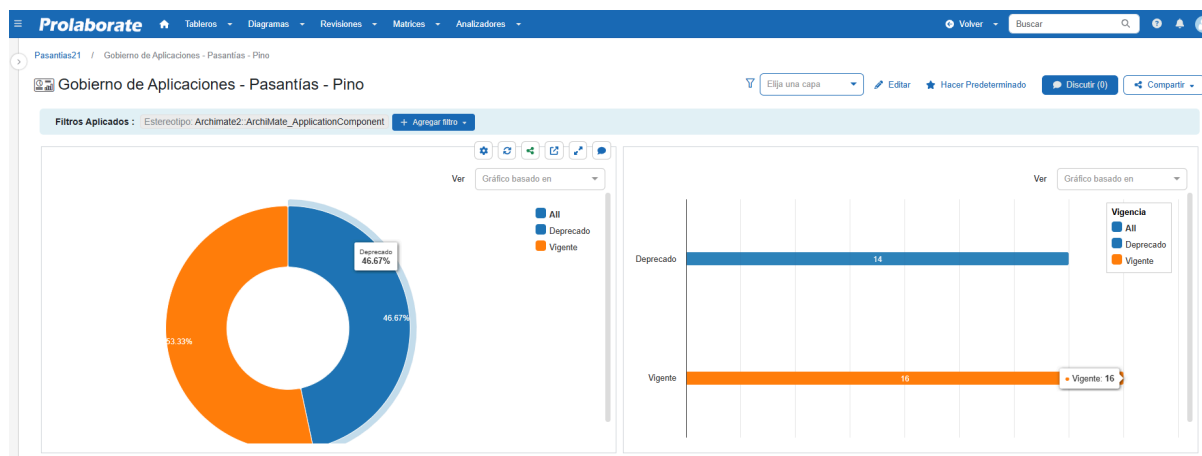
“Pino_Evidencias_Funcionales_Queries”

Visualización opcional en Prolaborate

Objetivo de la visualización

Como extensión opcional del ejercicio se construyó un dashboard en Prolaborate denominado:

Gobierno de Aplicaciones - Pasantías - Pino



El objetivo fue trasladar a una representación visual uno de los resultados obtenidos previamente mediante las consultas SQL desarrolladas y validadas en Enterprise Architect.

Se seleccionó como caso de uso la consulta **Q2-G — Vigencia — Grouped**, debido a que responde a una pregunta de negocio fácilmente interpretable mediante gráficos:

¿Cómo se distribuyen las aplicaciones entre los estados Vigente y Deprecado?

La consulta había sido validada previamente en Enterprise Architect con el siguiente resultado:

Vigente = 16

Deprecado = 14

Total = 30 aplicaciones

A partir de estos valores, construí dos visualizaciones complementarias:

- un gráfico de **dona**, orientado a mostrar la proporción relativa de cada estado;
- un gráfico de **barras**, orientado a comparar directamente las cantidades absolutas.

El dashboard final permite observar simultáneamente ambas perspectivas.

Consulta utilizada como referencia funcional

La consulta utilizada para obtener la distribución de vigencia fue:

```
SELECT
    tag.Value AS Vigencia,
    COUNT(DISTINCT app.Object_ID) AS Application_Count
FROM t_object app
INNER JOIN t_objectproperties tag
    ON tag.Object_ID = app.Object_ID
WHERE app.Object_Type = 'Class'
    AND app.Stereotype = 'ArchiMate_ApplicationComponent'
    AND tag.Property = 'Vigencia'
    AND tag.Value IN ('Vigente', 'Deprecado')
GROUP BY tag.Value
ORDER BY tag.Value;
```

Esta consulta representa la misma lógica utilizada durante la validación que hice en Enterprise Architect: identificar exclusivamente elementos de tipo aplicación, obtener el tagged value Vigencia y contabilizar aplicaciones únicas para cada estado. Antes de construir las visualizaciones se probó la misma consulta desde la opción de consultas SQL de Prolaborate. La ejecución fue correcta y devolvió:

Deprecado 14

Vigente 16

De esta manera se confirmó que Prolaborate accedía al mismo conjunto de información y que los resultados coincidían con los obtenidos previamente en EA.

Ejecutar Consulta - Level1 Query ✕

► Consulta

Vista Previa de la Tabla

Buscar

Vigencia !!	Application_Count !!
Deprecado	14
Vigente	16

Mostrando 1 a 2 de 2 entradas

<< < 1 > >>

✕ Cerrar 💾 Guardar Consulta

Configuración inicial mediante consulta SQL

La primera aproximación consistió en utilizar directamente la funcionalidad de creación de gráficos mediante SQL. Para el gráfico de dona se siguió el flujo:

Tableros

→ Crear

→ Agregar Widget

→ Gráficos

→ Dona

→ Saltar a Consulta

La documentación oficial de Prolaborate contempla específicamente la creación de gráficos Donut mediante **SQL Queries**, indicando que se puede saltar directamente a la configuración de consulta, introducir el SQL, ejecutar la consulta para validar el resultado y posteriormente avanzar a la configuración visual.

Al ejecutar Q2-G desde esta interfaz, Prolaborate devolvió correctamente los valores Deprecado = 14 y Vigente = 16. Sin embargo, se presentó una dificultad en la etapa de representación gráfica, que detallaré a continuación.

Dificultad encontrada

Aunque la consulta SQL se ejecutaba correctamente, la vista previa del componente gráfico no interpretaba directamente la estructura agregada:

Vigencia | Application_Count

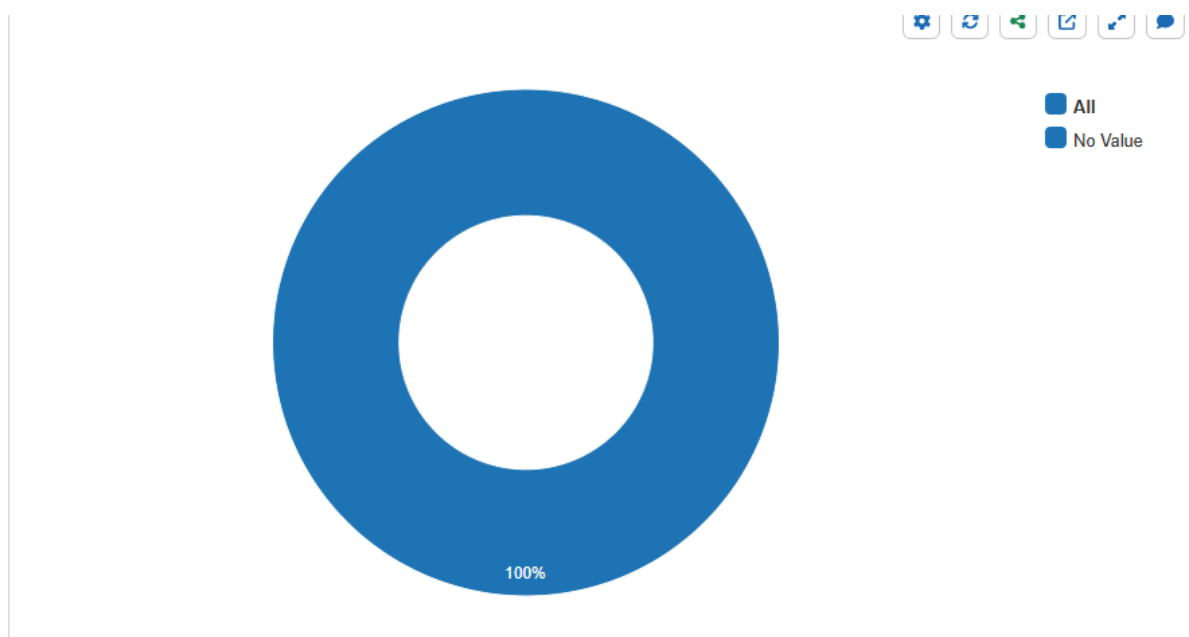
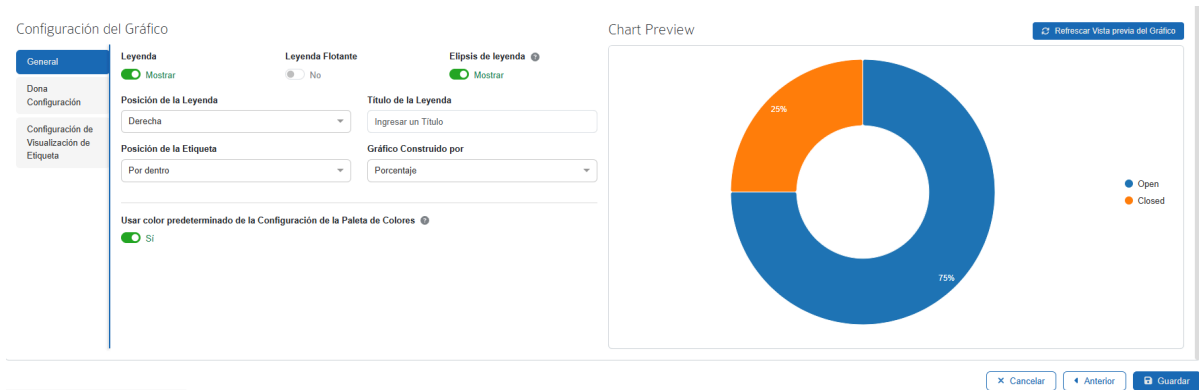
Durante la configuración del gráfico de dona, la vista previa continuaba mostrando datos genéricos de ejemplo, con categorías Open y Closed y porcentajes 75 % / 25 %. Luego, al guardar, mostraba un grafico Donut, solamente con el porcentaje al 100%

Esto generó inicialmente la duda de si la consulta había sido vinculada correctamente al gráfico. La ejecución manual de la consulta permitió descartar un problema en los datos, ya que Prolaborate sí recuperaba correctamente:

Deprecado 14

Vigente 16

El inconveniente se encontraba, por tanto, en la forma en que el componente gráfico esperaba recibir y organizar los datos.



Para comprender el formato esperado por Prolaborate se recurrió a la segunda alternativa documentada: **Configuración del Diseñador**.

La documentación oficial consultada fue:

<https://prolaborate.sparxsystems.com/resources/v5-documentation/build-donut-charts>

Esta Indica que el diseñador permite seleccionar el origen, tipo de elemento, estereotipo, propiedades o tagged values y generar automáticamente la consulta requerida por el gráfico. También permite visualizar posteriormente la query generada.

Se configuró el gráfico utilizando los mismos criterios funcionales que Q2-G:

Tipo:Class

Estereotipo: ArchiMate_ApplicationComponent

Propiedad: Valores Etiquetados → Vigencia

Valores: Vigente, Deprecado

Construir gráfico basado en: Valores Etiquetados → Vigencia

Al avanzar a la configuración de consulta, Prolaborate generó automáticamente una query cuya estructura era diferente de Q2-G. En lugar de devolver directamente dos filas agregadas, la consulta del Diseñador devolvía **una fila por aplicación**, incluyendo el valor de Vigencia en una columna utilizada por Prolaborate como Series. La ejecución mostró las **30 aplicaciones** del universo analizado, cada una clasificada como Vigente o Deprecado.

Luego de consultar con IA, y la documentación de estos resultados, comprendí que la consulta Q2-G era correcta desde el punto de vista SQL y devolvía los resultados requeridos, pero el componente visual de Prolaborate esperaba una estructura orientada a series para construir el gráfico automáticamente.

La consulta generada por el Diseñador mantuvo los mismos criterios funcionales de Q2-G y produjo la misma distribución final.

Configuración del gráfico de dona

Para el gráfico de dona se configuró **Vigencia** como criterio de agrupación. La documentación de Prolaborate establece que los Donut Charts permiten representar los resultados por valor o porcentaje y configurar elementos como leyenda, posición y etiquetas.

El gráfico final mostró:

Vigente 53,33 %

Deprecado 46,67 %

Estos porcentajes corresponden exactamente a:

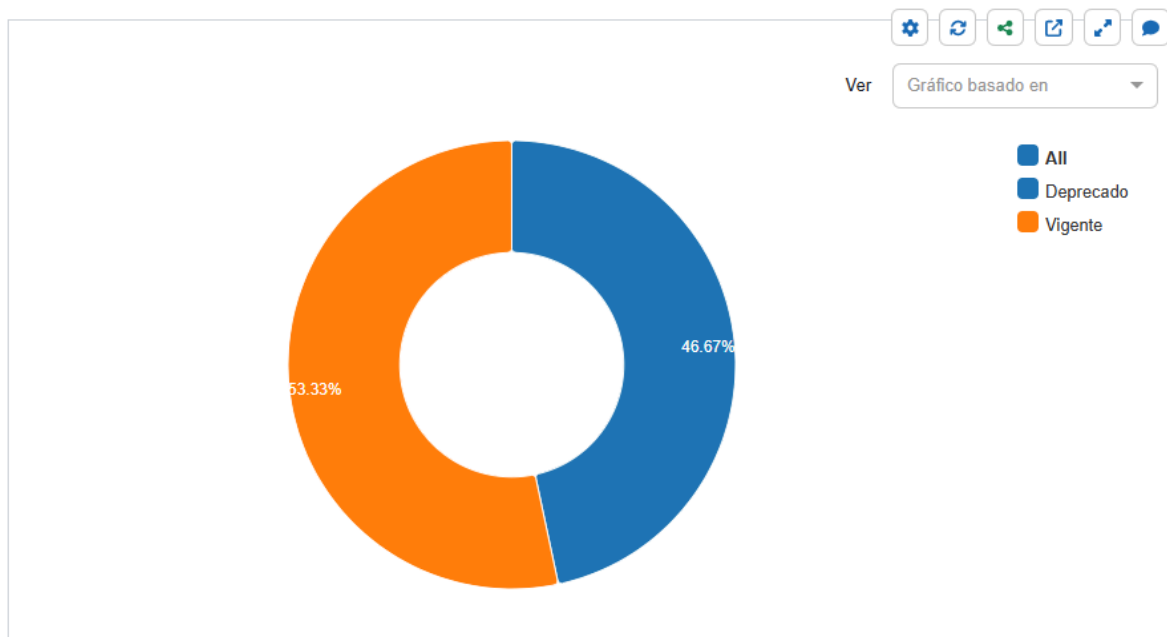
$16 / 30 = 53,33 \%$

$14 / 30 = 46,67 \%$

Por lo tanto, el gráfico reproduce correctamente la distribución obtenida mediante Q2-G.

El gráfico de dona responde principalmente a la pregunta:

¿Qué proporción del portfolio de aplicaciones se encuentra vigente y qué proporción se encuentra deprecada?



Configuración del gráfico de barras

Como segunda visualización, y con el objetivo de seguir explorando Prolaborate y sus alternativas, se incorporó un gráfico de barras para representar las cantidades absolutas. La configuración siguió los mismos criterios de aplicación y tagged value utilizados en Q2-G.

Esta configuración agregaba algunos parámetros. Para el gráfico se utilizó:

Eje Y: Valores Etiquetados → Vigencia

Eje X: Conteo

Agrupación adicional: Ninguna

La documentación de Prolaborate establece que, para gráficos de barras, el eje categórico puede construirse utilizando propiedades básicas o tagged values, mientras que la longitud de las barras puede calcularse mediante conteo o suma. La documentación oficial consultada fue:

<https://prolaborate.sparxsystems.com/resources/v5-documentation/build-stacked-bar-charts>

En este caso se utilizó **Conteo**, ya que el objetivo no era sumar una propiedad numérica, sino contabilizar cuántas aplicaciones pertenecen a cada estado. El gráfico final mostró:

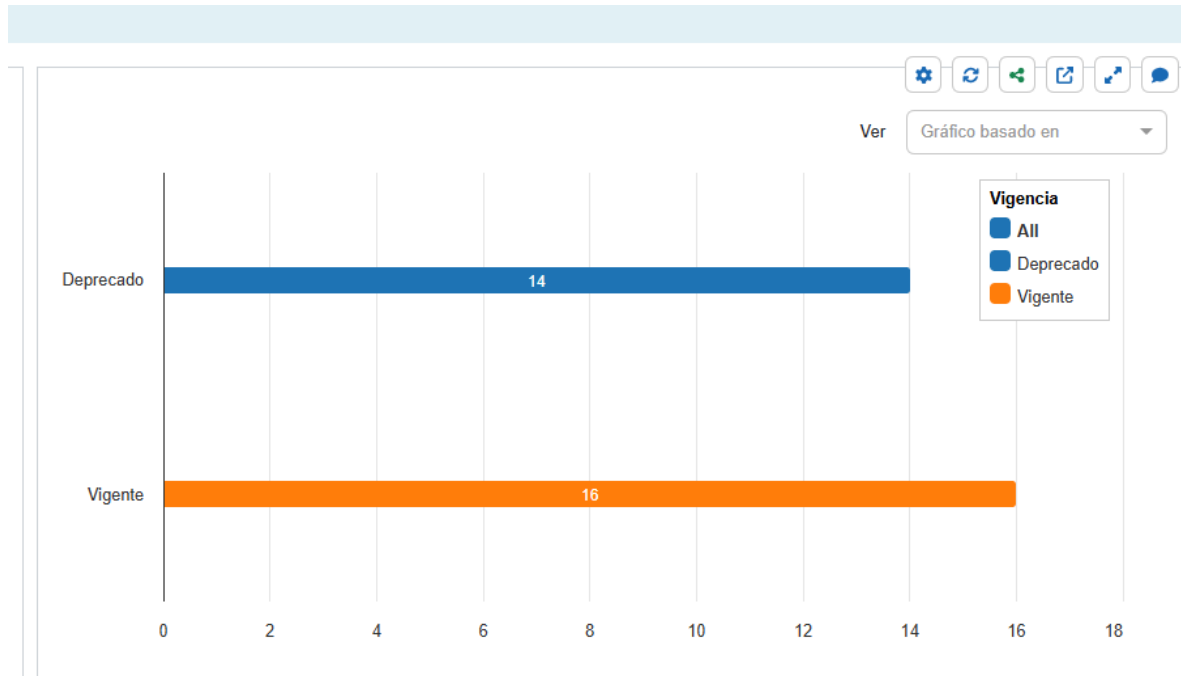
Deprecado = 14

Vigente = 16

La visualización responde directamente a la pregunta:

¿Cuántas aplicaciones existen actualmente en cada estado de vigencia?

A diferencia del gráfico de dona, que enfatiza la proporción relativa, el gráfico de barras permite comparar visualmente las cantidades absolutas.



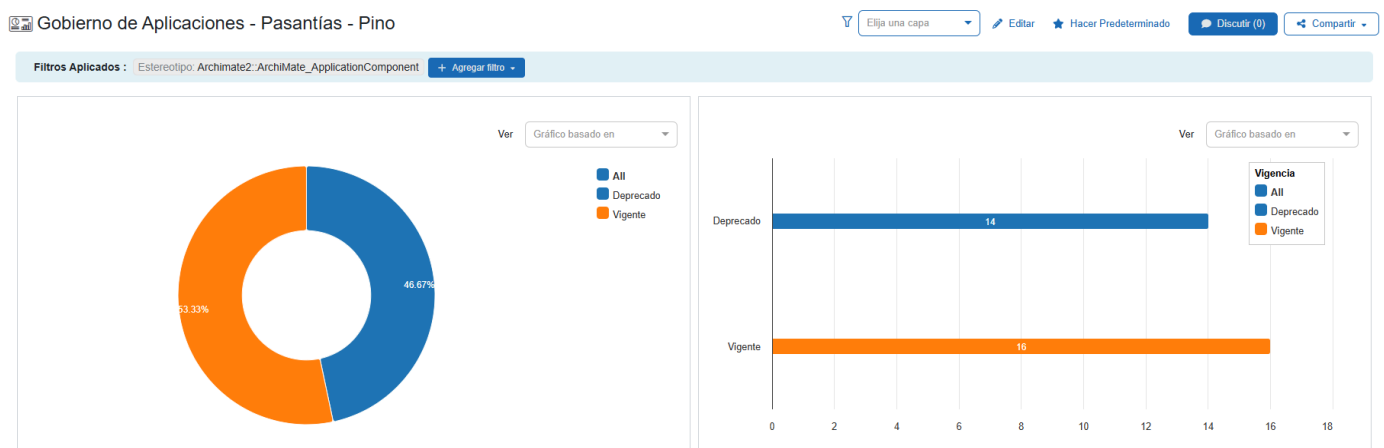
Resultado del dashboard

El dashboard final **Gobierno de Aplicaciones - Pasantías - Pino** contiene ambas visualizaciones.

El gráfico de dona muestra: Deprecado 46,67 %, Vigente 53,33 %

y el gráfico de barras muestra: Deprecado 14, Vigente 16

Ambas representaciones son consistentes entre sí y coinciden exactamente con los resultados previamente obtenidos mediante Q2-G en Enterprise Architect.



La correspondencia puede expresarse de la siguiente manera:

Fuente / Visualización	Deprecado	Vigente
Q2-G en Enterprise Architect	14	16
Q2-G ejecutada en Prolaborate	14	16
Dona en Prolaborate	46,67 %	53,33 %
Barras en Prolaborate	14	16

Esto permite confirmar que **el dashboard de Prolaborate representa correctamente la información obtenida a partir de la lógica de la query**, satisfaciendo el criterio de aceptación del punto opcional.

Evidencia funcional

La evidencia de funcionamiento se mantiene separada del presente informe para evitar mezclar la explicación técnica con grandes cantidades de capturas.

El documento:

[Pino_Evidencias_Funcionales_Queries](#)

Contiene las ejecuciones realizadas en Enterprise Architect, los resultados obtenidos, las comprobaciones de navegación, la visualización de tagged values y la validación de las relaciones utilizadas en el análisis de impacto.

Esta separación permite utilizar el presente documento como explicación técnica principal y el documento de evidencias como respaldo visual de los resultados.

Documentación técnica complementaria

Además de los documentos PDF preparados para presentar la solución y sus evidencias, durante el desarrollo mediante la metodología SDD se generaron distintos artefactos técnicos dentro del directorio Exercise_2_queries.

Estos archivos forman parte de la trazabilidad del proceso de desarrollo y permiten conservar información que, por su nivel de detalle técnico, no resulta conveniente incorporar íntegramente al presente informe.

Los agentes de IA utilizados mediante Gentle-AI trabajaron principalmente con archivos Markdown (.md) porque este formato resulta especialmente adecuado para un flujo de desarrollo basado en especificaciones. Al tratarse de archivos de texto plano, pueden ser leídos y actualizados de forma controlada por los distintos agentes, comparados mediante Git y utilizados como entrada para las fases posteriores del proceso SDD.

De esta manera, los archivos Markdown funcionan como **artefactos técnicos de trabajo y trazabilidad**, mientras que los documentos PDF constituyen la presentación final orientada a la revisión humana. Los .md no reemplazan a la evidencia funcional ni al informe principal, sino que conservan con mayor detalle las decisiones, diagnósticos y relaciones entre requisitos, consultas y resultados.

Los principales archivos generados para el Ejercicio 2 fueron:

[Pino_Exercise_2_Technical_Explainer.md](#)

Ubicado en Exercise_2_queries/docs/.

Contiene la explicación técnica detallada de la solución: tablas utilizadas del repositorio de Enterprise Architect, campos relevantes, relaciones entre tablas, filtros aplicados, criterio utilizado para identificar aplicaciones, estrategia de unicidad, interpretación de Categoría y Vigencia, identificación semántica de Base de Datos 28 y decisiones relacionadas con la compatibilidad de las consultas en EA SQL Search.

Este documento actúa como soporte técnico ampliado del presente informe.

[Pino_Exercise_2_SQLite_Diagnostics.md](#)

Ubicado en Exercise_2_queries/docs/.

Registra las comprobaciones realizadas directamente sobre el archivo .qea utilizando SQLite en modo read-only. Estos diagnósticos se utilizaron como mecanismo de validación secundaria antes de ejecutar las consultas en Enterprise Architect. Permitieron comprobar cantidades esperadas, posibles valores faltantes, duplicados y relaciones existentes sin modificar el repositorio. Los resultados obtenidos mediante SQLite fueron utilizados únicamente como oráculo diagnóstico; la validación funcional definitiva continuó realizándose en Enterprise Architect.

[Pino_Exercise_2_Evidence_Index.md](#)

Ubicado en Exercise_2_queries/docs/.

Funciona como índice de trazabilidad de las cinco consultas desarrolladas: Q1-C, Q1-L, Q2-G, Q3-C y Q3-L. Para cada consulta relaciona el interrogante que responde, su ubicación dentro del archivo SQL, el resultado esperado, el resultado obtenido en Enterprise Architect, las comprobaciones realizadas contra el modelo y la evidencia funcional correspondiente. Su objetivo es permitir reconstruir la cadena:

pregunta → consulta SQL → resultado → validación en EA → evidencia

[Pino_Exercise_2_Prolaborate_Q2.md](#)

Ubicado en Exercise_2_queries/docs/.

Documenta técnicamente el punto opcional realizado en Prolaborate. Registra la utilización de Q2-G como referencia funcional, la ejecución directa de la consulta con los resultados Vigente=16 y Deprecado=14, la dificultad encontrada al intentar representar directamente su estructura agregada y la resolución mediante Designer Configuration. También documenta que los widgets finales utilizaron la estructura orientada a Series generada por el Diseñador, preservando exactamente la lógica y los resultados previamente validados en Enterprise Architect.

[Pino_Exercise_2_EA_Governance_Queries.sql](#)

Ubicado en Exercise_2_queries/queries/.

Es el archivo SQL principal de la entrega y contiene exactamente las cinco consultas finales desarrolladas: Q1-C, Q1-L, Q2-G, Q3-C y Q3-L.

El archivo se utiliza como contenedor reproducible de las consultas. Cada sentencia fue ejecutada individualmente en Enterprise Architect SQL Search y todas son exclusivamente de lectura. Esta separación entre documentación de presentación, documentación técnica y código SQL permite conservar una entrega legible para el evaluador sin perder la trazabilidad completa del proceso de análisis, construcción y validación.